Name: Dishita Bansal Roll No.:1024030365 Batch:2C24

DSA ASSIGNMENT-1

QUES 1) Develop a Menu driven program to demonstrate the following operations of Arrays

——MENU——-

1.CREATE

2.DISPLAY

3.INSERT

4.DELETE

5.LINEAR SEARCH

6.EXIT

Solution :

```
#include<iostream>
using namespace std;
int arr[1000];
int n;
void display(){
    for(int i = 0;i<n;i++){
        cout<<arr[i];
    }
}
main(){
    int option;
    int choice;

    cout<<"enter the size of array: ";
    cin>>n;
    for(int i = 0;i<n;i++){
        cout<<"Element "<<i+1<<":";
        cin>>arr[i];
    }
    display();
    do{
    cout<<"\nOption 1 : Insert element in array";
    cout<<"\nOption 2 : Delete an element in array";
    cout<<"\nOption 3 : Linear search ";
    cout<<"\nOption 4 : Exit";
    cout<<"\nWhich option you would choose ";
    cin>>option;
    switch(option){
        case 1 :
        int element,pos;
```

```
        cout<<"enter the element you want to insert :";
        cin>>element;
        cout<<"enter the pos where you want to insert element :";
        cin>>pos;
        for(int i = n-1;i>=pos-1;i--){
            arr[i+1] = arr[i];

        }
        arr[pos-1]=element;
        n++;
        display();
        break;
        case 2 :
        int index;
        display();
        cout<<"\nenter the index of the element where you want to wish to
delete : ";
        cin>>index;
        if(index > n +1 ){
            cout<<"deletion not possible";
        }
        else{
            for(int i = index-1;i<n-1;i++){
            arr[i]=arr[i+1];
            }
        }
        n--;
        display();
        break;
        case 3 :
        int item;
        cout<<"Enter the element you want to search ";
        cin>>item;
        for(int i = 0;i<n;i++){
            if(arr[i]==item){
                cout<<"element found at position :"<<i+1;
            }
            else{
                cout<<"element didnt found";
            }
        }

    }
    cout<<"\npress 0 for exit..press 1 for further ";
    cin>>choice;
}while(choice == 1);
}
```

Output :

QUES 2) Design the logic to remove the duplicate elements from an Array and after the deletion the array should contain the unique elements.

SOLUTION :

```
#include<iostream>
using namespace std;
int main(){
    int n;
    int pos;
    int arr[1000];
    cout<<"Enter the size of array : ";
    cin>>n;
    for(int i = 0;i < n;i++){
        cout<<"Element "<<i+1<<":";6
        cin>>arr[i];
    }
    //sorting
    for(int i = 0;i<n-1;i++){
        for(int j = 0;j<n-i-1;j++){
            if(arr[j]>arr[j+1]){
               int  temp=arr[j];
               arr[j]=arr[j+1];
```

```
                arr[j+1]=temp;
            }
        }
    }
    for(int i = 1;i<n;){
        if(arr[i-1]==arr[i]){
           for(int j = i; j<n-1;j++){
                arr[j]=arr[j+1];

            } n--;
        }
        else{
            i++;
        }

    }
    for(int i = 0;i<n;i++){
        cout<<arr[i];
    }
}
```

OUTPUT:

```
 tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter the size of array : 6
Element 1:3
Element 2:4
Element 3:8
Element 4:3
Element 5:6
Element 6:7
34678
```

QUES 3)Predict the Output of the following program int main() {

int i; int arr[5] = {1}; for (i = 0; i < 5; i++) printf("%d",arr[i]); return 0; }

```
 tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
10000
```

QUES 4) Implement the logic to a. Reverse the elements of an array b. Find the matrix multiplication c. Find the Transpose of a Matrix

```
#include<iostream>
using namespace std;
main(){
    int option;
    cout<<"Option 1 : Reverse the array ";
```

```
    cout<<"\nOption 2 : Matrix multiplication";
    cout<<"\nOption 3 : Transpose of matrix ";
    cout<<"\nEnter the option";
    cin>>option;
    switch(option){
        case 1 :{
        int arr[1000];
        int n;
        cout<<"Enter the size of array";
        cin>>n;
        for(int i = 0;i<n;i++){
            cout<<"element "<<i+1<<":";
            cin>>arr[i];
        }
        int start = 0;
        int end = n-1;
        while(start < end){
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }
        for(int i = 0;i<n;i++){
            cout<<arr[i];
        }

        break;}
        case 2:{
        int r,c;
        int r1,c1;
        int arr1[100][100],arr2[100][100];
        int arr3[100][100];

        cout<<"Enter no of rows in matrix 1: ";
        cin>>r;
        cout<<"Enter no of columns in matrix 1: ";
        cin>>c;
        cout<<"Enter elements for matrix 1 \n";
        for(int i = 0;i<r;i++){
            for(int j = 0;j<c;j++){
                cout<<"Enter element at "<<"["<<i<<"]"<<"["<<j<<"]";
                cin>>arr1[i][j];
            }
        }
        cout<<"Enter no of rows in matrix 2 : ";
        cin>>r1;
        cout<<"Enter no of columns in matrix 2 : ";
```

```
        cin>>c1;
        cout<<"Enter elements for matrix 2 \n";
        for(int i = 0;i<r1;i++){
            for(int j = 0;j<c1;j++){
                cout<<"Enter element at "<<"["<<i<<"]"<<"["<<j<<"]";
                cin>>arr2[i][j];
            }
        }
        if(c!=r1){
            cout<<"Multiplication cant be done as no of columns of first
matrix is not same as no of rows in second matrix";
        }
        for(int i = 0;i<r;i++){
            for(int j = 0;j<c1;j++){
                arr3[i][j]=0;

            }
        }
        for(int i = 0;i<r;i++){
            for(int j = 0;j<c1;j++){
                for(int k =0;k<c1;k++){
                    arr3[i][j]+=arr1[i][k]*arr2[k][j];
                }
            }
        }
        cout<<"resultant matrix";
         for(int i = 0;i<r;i++){
            for(int j = 0;j<c1;j++){
                cout<<arr3[i][j];
                cout<<" ";

            }cout<<endl;
        }
        break;}
        case 3 :
        {
        int arr4[100][100];
        int r3,c3;
        cout<<"Enter the no of rows : ";
        cin>>r3;
        cout<<"Enter the no of columns : ";
        cin>>c3;
        cout<<"Enter elements for matrix \n";
        for(int i = 0;i<r3;i++){
            for(int j = 0;j<c3;j++){
                cout<<"Enter element at "<<"["<<i<<"]"<<"["<<j<<"]";
                cin>>arr4[i][j];
            }
```

```
        }
        for(int i = 0;i<c3;i++){
            for(int j = 0;j<r3;j++){
                cout<<arr4[j][i];
                cout<<" ";
            }
            cout<<endl;
        }

    }}
}
```

Output:

```
 ($?) { .\ques4 }
Option 1 : Reverse the array
Option 2 : Matrix multiplication
Option 3 : Transpose of matrix
Enter the option1
Enter the size of array5
element 1:1
element 2:2
element 3:3
element 4:4
element 5:5
54321
```

```
 ($?) { .\ques4 }
Option 1 : Reverse the array
Option 2 : Matrix multiplication
Option 3 : Transpose of matrix
Enter the option2
Enter no of rows in matrix 1: 2
Enter no of columns in matrix 1: 2
Enter elements for matrix 1
Enter element at [0][0]1
Enter element at [0][1]2
Enter element at [1][0]3
Enter element at [1][1]4
Enter no of rows in matrix 2 : 2
Enter no of columns in matrix 2 : 2
Enter elements for matrix 2
Enter element at [0][0]1
Enter element at [0][1]2
Enter element at [1][0]
3
Enter element at [1][1]4
resultant matrix
7 10
15 22
```

```
 ($?) { .\ques4 }
Option 1 : Reverse the array
Option 2 : Matrix multiplication
Option 3 : Transpose of matrix
Enter the option3
Enter the no of rows : 3
Enter the no of columns : 3
Enter elements for matrix
Enter element at [0][0]1
Enter element at [0][1]2
Enter element at [0][2]3
Enter element at [1][0]4
Enter element at [1][1]5
Enter element at [1][2]6
Enter element at [2][0]7
Enter element at [2][1]8
Enter element at [2][2]9
1 4 7
2 5 8
3 6 9
```

5) Write a program to find sum of every row and every column in a two-dimensional array.

```
#include<iostream>
using namespace std;
main(){
    int arr[100][100];
    int r,c;
    cout<<"Enter the no of  rows";
    cin>>r;
    cout<<"Enter the no of columns";
    cin>>c;
    for(int i = 0;i<r;i++){
        for(int j =0;j<c;j++){
             cout<<"Enter element at "<<"["<<i<<"]"<<"["<<j<<"] :";
            cin>>arr[i][j];
        }
    }
     for(int i = 0;i<r;i++){
        for(int j =0;j<c;j++){

          cout<<arr[i][j];
          cout<<" ";
        }cout<<endl;
    }
    int option;
    cout<<"Option 1 : Row-wise sum";
    cout<<"\nOption 2 : column wise sum";
    cout<<"\nOption 3 : Sum of whole matrix";
    cout<<"Enter the option: ";
    cin>>option;
    switch(option){
        case 1:
    int sum ;
```

```
    for(int i = 0;i<r;i++){
       sum = 0;
        for(int j =0;j<c;j++){
        sum+=arr[i][j];
        }
        cout<<"Sum of "<<i+1<<"row : "<<sum;
        cout<<endl;

    }
    break;
    case 2 :
      int sum2;
    for(int j = 0;j<c;j++){
       sum2 = 0;
        for(int i =0;i<r;i++){
        sum2+=arr[i][j];
        }
        cout<<"Sum of "<<j+1<<"column : "<<sum2;
        cout<<endl;}
        break;
    case 3:
    int sum1 = 0;
    for(int i =0;i<r;i++){
        for(int j = 0;j<c;j++){
            sum1+=arr[i][j];
        }
    }
    cout<<"Sum of whole matrix : "<<sum1;
    }
}
```

Output:

```
 tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter the no of  rows3
Enter the no of columns3
Enter element at [0][0] :1
Enter element at [0][1] :2
Enter element at [0][2] :3
Enter element at [1][0] :4
Enter element at [1][1] :5
Enter element at [1][2] :6
Enter element at [2][0] :7
Enter element at [2][1] :8
Enter element at [2][2] :9
1 2 3
4 5 6
7 8 9
Option 1 : Row-wise sum
Option 2 : column wise sum
Option 3 : Sum of whole matrixEnter the option: 1
Sum of 1row : 6
Sum of 2row : 15
Sum of 3row : 24
```

Name: Dishita Bansal Roll No.:1024030365 Batch:2C24

```
Option 1 : Row-wise sum
Option 2 : column wise sum
Option 3 : Sum of whole matrixEnter the option: 2
Sum of 1column : 12
Sum of 2column : 15
Sum of 3column : 18
```

```
Option 1 : Row-wise sum
Option 2 : column wise sum
Option 3 : Sum of whole matrixEnter the option: 3
Sum of whole matrix : 45
```

Name: Dishita Bansal Roll No.1024030365 Batch:2C24

DSA ASSIGNMENT-2

QUES 1) Implement the Binary search algorithm regarded as a fast search algorithm with run-time complexity of O(log n) in comparison to the Linear Search.

SOLUTION :

```
#include<iostream>
using namespace std;
main(){
    int arr[100];
    int n ;
    cout<<"enter the size of array : ";
    cin>>n;
    for(int i=0;i<n;i++){
        cout<<"Enter element "<<i+1;
        cin>>arr[i];
    }
    for(int i=0;i<n-1;i++){
        for(int j = 0;j<n-i-1;j++){
            if(arr[j]>arr[j+1]){
                int temp = arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=temp;

            }
        }
    }
    int item;
    cout<<"Enter the no you want to search";
    cin>>item;
    int low = 0;
    int high = n-1;
    int mid;
    int pos = -1 ;
    while(low<=high){
        mid = (low+high)/2;
        if(arr[mid]==item){
            pos = mid;
            break;
        }
        else if ( arr[mid]<item){
            low = mid+1;

        }
```

```
        else{
            high = mid + 1 ;

        }
    }
    if(pos!=-1){
        cout<<"Element found at position "<<pos+1<<endl;

    }
    else{
        cout<<"Element not found ";
    }


}
```

Output:

```
ques1 } ; if ($?) { .\ques1 }
enter the size of array : 6
Enter element  1:2
Enter element  2:4
Enter element  3:6
Enter element  4:8
Enter element  5:9
Enter element  6:7
Enter the no you want to search :8
Element found at position :5
```

Ques2) Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order. Code the Bubble sort with the following elements:

Solution :

```
#include<iostream>
using namespace std;
main(){
    int arr[7]={64,34,25,12,22,11,90};
    for(int i = 0;i<=6;i++){
       for(int j =0;j<=6-i;j++){
        if(arr[j]>arr[j+1]){
        int temp = arr[j];
        arr[j]=arr[j+1];
        arr[j+1]=temp;

        }
```

```
    }
}
    for(int i = 0; i<7;i++){
        cout<<arr[i];
        cout<<endl;
    }

}
```

Output:

```
rFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
11
12
22
25
34
64
90
```

Ques 3) Given an array of n-1 distinct integers in the range of 1 to n, find the missing number in it in a Sorted Array (a) Linear time (b) Using binary search.

Solution:

```
#include<iostream>
using namespace std;
main(){
    int n ;
    int arr[100];
    cout<<"enter the size of array : ";
    cin>>n;

    for(int i = 0;i<n-1;i++){
        cout<<"Enter element from (1-n) range : " ;
        cin>>arr[i];
    }
    //sorting the above array
    for(int i = 0;i<n-2;i++){
        for(int j = 0;j<n-i-2;j++){
         if(arr[j]>arr[j+1]){
            int temp = arr[j];
            arr[j]=arr[j+1];
            arr[j+1]=temp;
         }
        }
    }
```

```
    for(int i = 0;i<n-1;i++){
        cout<<arr[i];
    }
    //by linear search
    int missing;
    for(int i =0;i<n;i++){
        if(arr[i]!=i+1){
         missing = i+1;
            break;
        }
    }
    cout<<"\nmissing no was : "<<missing;
    //by binary search
    int left = 0;
    int right = n-2;
    int mid;
    while(left<=right){
       mid = (left+right)/2;
       if(arr[mid]==mid+1){
        left = mid+1;
       }
       else{
        right = mid - 1;
       }

    }
cout<<"\nmissing no is :"<<left+1;
}
```

Ouput:

```
ques3 } ; if ($?) { .\ques3 }
enter the size of array : 6
Enter element from (1-n) range : 1
Enter element from (1-n) range : 3
Enter element from (1-n) range : 5
Enter element from (1-n) range : 6
Enter element from (1-n) range : 4
13456
missing no was : 2
missing no is :2
```

Ques 4) String Related Programs (a) Write a program to concatenate one string to another string. (b) Write a program to reverse a string. (c) Write a program to delete all the vowels from the string. (d) Write a program to sort the strings in alphabetical order. (e) Write a program to convert a character from uppercase to lowercase.

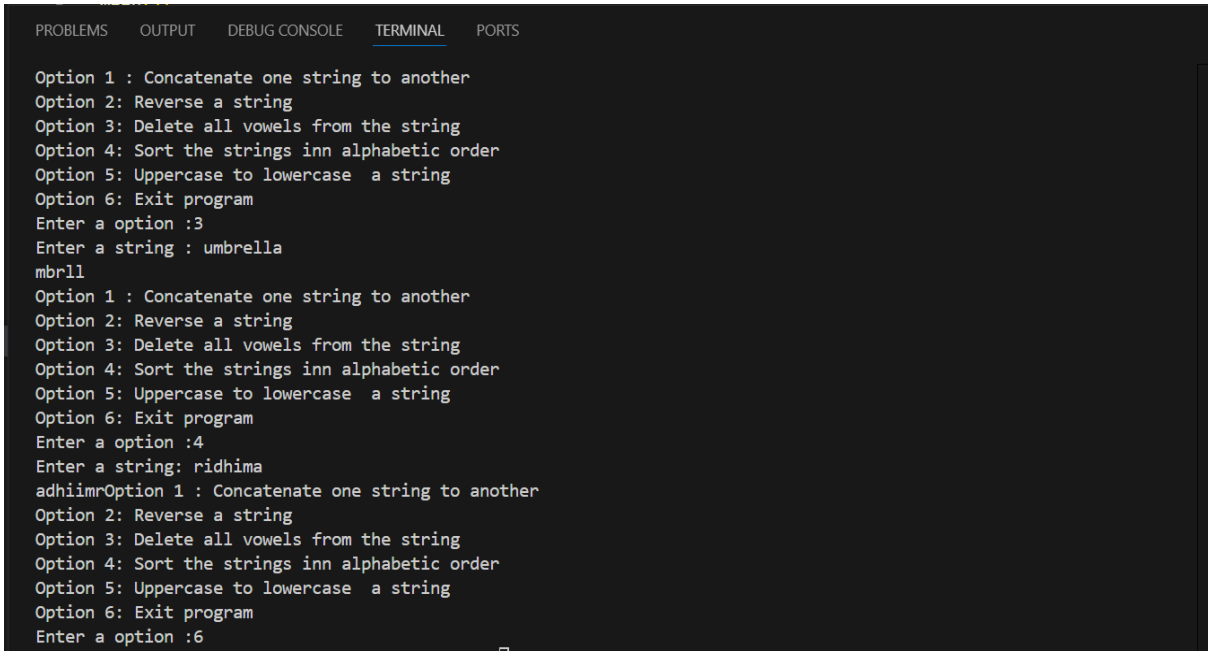
Solution:

```
#include<iostream>
using namespace std;
main(){
    int option;
    cout<<"Option 1 : Concatenate one string to another ";
    cout<<"\nOption 2: Reverse a string";
    cout<<"\nOption 3: Delete all vowels from the string";
    cout<<"\nOption 4: Sort the strings inn alphabetic order ";
    cout<<"\nOption 5: Uppercase to lowercase  a string";
    cout<<"\nEnter a option :";
    cin>>option;
    switch(option){
        case 1 :{
            string str1;
            string str2;
            cout<<"Enter one string:";
            getline(cin,str1);
            cout<<"Enter second string:";
            getline(cin,str2);
            str1=str1+ " "+ str2;
            cout<<"Concatenated string : "<<str1;
            break;}
         case 2 :
        { string str3;
         cout<<"Enter a string";
         cin>>str3;

         for(int i = str3.length()-1 ;i>=0;i--){
            cout<<str3[i];
         } break; }

         case 3:{
         string str4;
         cout<<"Enter a string : ";
         cin>>str4;
         for(int i =0;i<str4.length();i++){
            if(str4[i]!= 'a' && str4[i]!='e' && str4[i]!='i'&&
str4[i]!='o'&&str4[i]!='u' )
              cout<<str4[i];
        }
      break;}

        case 4 :{
        string str5;
        cout<<"Enter a string: ";
        cin>>str5;
```

```
        for(int i= 0;i<str5.length()-1;i++){
            for(int j = 0;j<str5.length()-i-1;j++){
            if(str5[j]>str5[j+1]){
          int  temp = str5[j];
               str5[j]=str5[j+1];
               str5[j+1]=temp;
            }
        }
        }
        for(int i = 0;i<str5.length();i++){
                 cout<<str5[i];
        }
    }

}
}
```

Output:

```
ues4 } ; if ($?) { .\ques4 }
Option 1 : Concatenate one string to another
Option 2: Reverse a string
Option 3: Delete all vowels from the string
Option 4: Sort the strings inn alphabetic order
Option 5: Uppercase to lowercase  a string
Option 6: Exit program
Enter a option :1
Enter one string:ridhima
Enter second string:goel
Concatenated string : ridhima goel
Option 1 : Concatenate one string to another
Option 2: Reverse a string
Option 3: Delete all vowels from the string
Option 4: Sort the strings inn alphabetic order
Option 5: Uppercase to lowercase  a string
Option 6: Exit program
Enter a option :2
Enter a stringridhi
ihdir
```



Ques 6 Write a program to implement the following operations on a Sparse Matrix, assuming the matrix is represented using a triplet. (a) Transpose of a matrix. (b) Addition of two matrices. (c) Multiplication of two matrices.

Solution

```
#include<iostream>
using namespace std;
main(){
    int arr[100][100];
    int r;
    int c;
    cout<<"Enter no of rows :";
    cin>>r;
    cout<<"Enter no of columns :";
    cin>>c;
    for(int i = 0;i<r;i++){
        for(int j = 0;j<c;j++){
            cout<<"Enter element "<<"["<<i<<"]"<<"["<<j<<"] :";
            cin>>arr[i][j];
        }
    }
    int sparse[100][3];
    int k = 1;
    for(int i = 0 ;i<r;i++){
        for(int j = 0;j<c;j++){
            if(arr[i][j]!=0){
               sparse[k][0]=i;
```

```
                sparse[k][1]=j;
                sparse[k][2]=arr[i][j];
                k++;
            }
        }
    }
    sparse[0][0]=r;
    sparse[0][1]=c;
    sparse[0][2]=k-1;
   for(int i = 0;i<k;i++){
    for(int j =0;j<3;j++){
        cout<<sparse[i][j];
    }
    cout<<endl;
   }


}
```

Name: Dishita Bansal Roll No.1024030365 Batch:2C24

Assignment-3

Ques 1. Develop a menu driven program demonstrating the following operations on a Stack using array:

(i) push(), (ii) pop(), (iii) isEmpty(), (iv) isFull(), (v) display(), and (vi) peek().

Code:

```
#include<iostream>

using namespace std;

const int MAX = 5;

class stack{
private:
    int arr[MAX];
    int top;
public:
    stack(){
        top = -1;
    }
    bool isEmpty(){
        return (top == -1);
    }
    bool isFull(){
        return (top == MAX-1);
    }
    void push(int a){
        if (isFull()) {
            cout << "Stack Overflow! Cannot push " << a << endl;
        } else {
            arr[++top] = a;
            cout << a << " pushed into stack." << endl;
        }
    }
    void pop(){
        if (isEmpty()){
            cout<<"Stack Underflow! Cannot pop"<<endl;
        }
        else{
            cout << arr[top--] << " popped from stack." << endl;
        }
    }
    void peek(){
        if(isEmpty()){
            cout<<"Stack is Empty ";
```

```
        }
        else{
            cout<<"The peak element is: "<<arr[top]<<endl;
        }
    }
    void display(){
        if (isEmpty()) {
            cout << "Stack is empty" << endl;
        } else {
            cout << "Stack elements are : ";
            for (int i = top; i >= 0; i--) {
                cout << arr[i] << " ";
            }
            cout << endl;
        }
    }
};

int main(){
    stack s;
    int choice ,value;
    do{
        cout << "\n--- Stack Menu ---" << endl;
        cout << "1. Push" << endl;
        cout << "2. Pop" << endl;
        cout << "3. Peek" << endl;
        cout << "4. isEmpty" << endl;
        cout << "5. isFull" << endl;
        cout << "6. Display" << endl;
        cout << "7. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;
        switch(choice){
            case 1:
                cout << "Enter value to push: ";
                cin >> value;
                s.push(value);
                break;
            case 2:
                s.pop();
                break;
            case 3:
                s.peek();
                break;
            case 4:
                if (s.isEmpty())
                    cout << "Stack is empty." << endl;
                else
```

```
                    cout << "Stack is not empty." << endl;
                break;
            case 5:
                if (s.isFull())
                    cout << "Stack is full." << endl;
                else
                    cout << "Stack is not full." << endl;
                break;
            case 6:
                s.display();
                break;
            case 7:
                cout << "Exiting program." << endl;
                break;
            default:
                cout << "Invalid choice! Try again." << endl;
        }
    } while (choice != 7);

    return 0;
}
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS


--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. isEmpty
5. isFull
6. Display
7. Exit
Enter your choice: 1
Enter value to push: 6
6 pushed into stack.

--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. isEmpty
5. isFull
6. Display
7. Exit
Enter your choice: 6
Stack elements are : 6 4 3

--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. isEmpty
5. isFull
6. Display
7. Exit
Enter your choice:
```

Ques2. Given a string, reverse it using STACK. For example “DataStructure” should be output as

“erutcurtSataD.”

Code:

```
#include <iostream>

using namespace std;

class Stack {
private:
    char *arr;
    int top;
    int capacity;

public:
    Stack(int size) {
        capacity = size;
        arr = new char[capacity];
        top = -1;
    }

    void push(char c) {
        if (top == capacity - 1) {
            cout << "Stack Overflow!" << endl;
            return;
        }
        arr[++top] = c;
    }

    char pop() {
        if (top == -1) {
            cout << "Stack Underflow!" << endl;
            return '\0';
        }
        return arr[top--];
    }
    bool isEmpty() {
        return top == -1;
    }
};

int main() {
    string str;
    cout << "Enter a string: ";
    getline(cin, str);

    Stack s(str.length());
```

```
    for (int i = 0; i < str.length(); i++) {
        s.push(str[i]);
    }

    string reversed = "";
    while (!s.isEmpty()) {
        reversed += s.pop();
    }

    cout << "Reversed string: " << reversed << endl;

    return 0;
}
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\HP\OneDrive\Desktop\CODING> cd "c:\Users\HP\OneDrive\Desktop\CO
2 }
Enter a string: DataStructure
Reversed string: erutcurtSataD
PS C:\Users\HP\OneDrive\Desktop\CODING\c\c++>
```

Ques3. Write a program that checks if an expression has balanced parentheses.

Code:

```
#include<iostream>
using namespace std;
const int MAX = 100;
class stack{
    private:
    int arr[MAX];
    int top;
    public:
    Stack() {
        top = -1;
     }

    bool isEmpty() {
         return top == -1;
        }
    bool isFull() {
        return top == MAX - 1;
    }

    void push(char ch) {
        if (isFull()) {
```

```
            cout << "Stack Overflow!" << endl;
            return;
        }
        else{
        arr[++top] = ch;
        }
    }

    char pop() {
        if (isEmpty()) {
            cout << "Stack Underflow!" << endl;
            return '\0';
        }
        return arr[top--];
    }

    char peek() {
        if (!isEmpty())
            return arr[top];
        return '\0';
    }
};
bool isbalanced(string exp){
    stack s;

    for (int i = 0; i < exp.length(); i++) {
        char ch = exp[i];

        if (ch == '(' || ch == '{' || ch == '[') {
            s.push(ch);
        }
         else if (ch == ')' || ch == '}' || ch == ']') {
            if (s.isEmpty())
                return false;

            char top = s.pop();
            if ((ch == ')' && top != '(') ||
                (ch == '}' && top != '{') ||
                (ch == ']' && top != '[')) {
                return false;
            }
        }
    }

    return s.isEmpty();
}
int main() {
    string exp;
```

```
    cout << "Enter an expression: ";
    cin >> exp;

    if (isbalanced(exp))
        cout << "Parentheses are balanced." << endl;
    else
        cout << "Parentheses are NOT balanced." << endl;

    return 0;
}
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\HP\OneDrive\Desktop\CODING\c\c++>

                                          > cd "c:\Users\HP\OneDrive\Desktop\
\stack3 }
Enter an expression: {AB+(CD-*)}
Parentheses are NOT balanced.
PS C:\Users\HP\OneDrive\Desktop\CODING\c\c++>
```

Ques4. Write a program to convert an Infix expression into a Postfix expression.

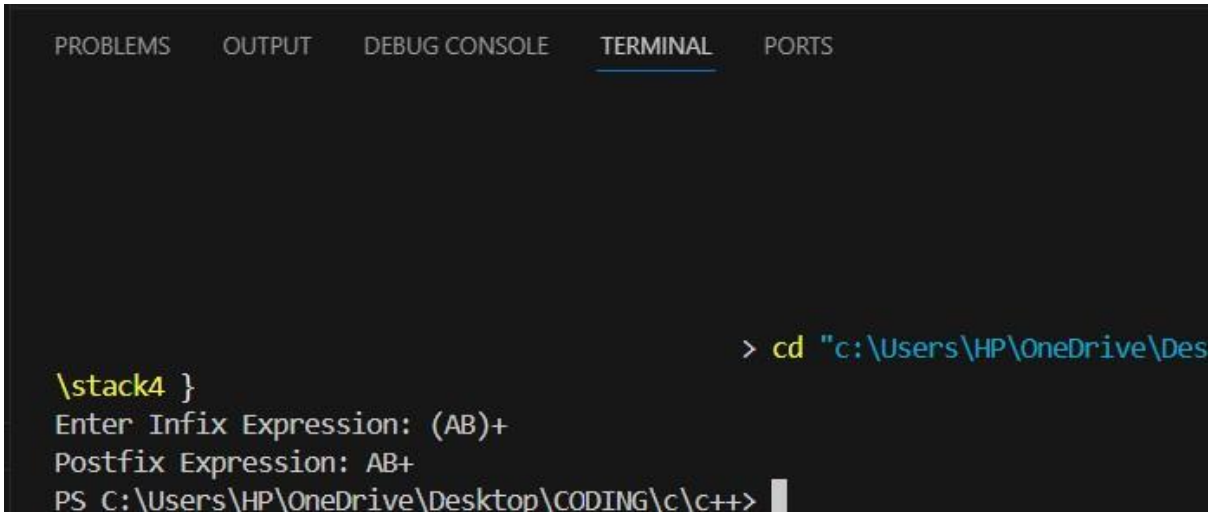
Code:

```
#include <iostream>
#include <cstring>
using namespace std;
const int MAX = 100;

class Stack {
    private:
        char arr[MAX];
        int top;
    public:
        Stack() {
            top = -1;
        }

        bool isEmpty() {
            return top == -1;
        }

        bool isFull() {
            return top == MAX - 1;
```

```
        }

        void push(char c) {
            if (!isFull()) {
                arr[++top] = c;
            } else {
                cout << "Stack Overflow!" << endl;
            }
        }

        char pop() {
            if (!isEmpty()) {
                return arr[top--];
            } else {
                cout << "Stack Underflow!" << endl;
                return '\0';
            }
        }

        char peek() {
            if (!isEmpty()) {
                return arr[top];
            } else {
                return '\0';
            }
        }
};

int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    if (op == '^') return 3;
    return 0;
}

bool isOperator(char c) {
    return (c == '+' || c == '-' || c == '*' || c == '/' || c == '^');
}

void infixToPostfix(char infix[]) {
    Stack s;
    char postfix[MAX];
    int k = 0;

    for (int i = 0; i < strlen(infix); i++) {
        char c = infix[i];

        // If operand, add to postfix
```

```
        if ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z') || (c >= '0' && c
<= '9')) {
            postfix[k++] = c;
        }
        else if (c == '(') {
            s.push(c);
        }
        else if (c == ')') {
            while (!s.isEmpty() && s.peek() != '(') {
                postfix[k++] = s.pop();
            }
            s.pop();
        }
        else if (isOperator(c)) {
            while (!s.isEmpty() && precedence(s.peek()) >= precedence(c)) {
                postfix[k++] = s.pop();
            }
            s.push(c);
        }
    }

    while (!s.isEmpty()) {
        postfix[k++] = s.pop();
    }

    postfix[k] = '\0';
    cout << "Postfix Expression: " << postfix << endl;
}

int main() {
    char infix[MAX];
    cout << "Enter Infix Expression: ";
    cin >> infix;

    infixToPostfix(infix);
    return 0;
}
```

Output:



Ques 5. Write a program for the evaluation of a Postfix expression.

Code:

```
#include <iostream>
#include <cstring>
#include <cctype>
using namespace std;

#define MAX 100

class Stack {
    int arr[MAX];
    int top;

public:
    Stack() { top = -1; }

    void push(int x) {
        if (top == MAX - 1) {
            cout << "Stack Overflow\n";
            return;
        }
        arr[++top] = x;
    }

    int pop() {
        if (top == -1) {
            cout << "Stack Underflow\n";
            return -1;
        }
        return arr[top--];
    }
```

```
    bool isEmpty() {
        return (top == -1);
    }

    int peek() {
        if (top == -1) {
            cout << "Stack Empty\n";
            return -1;
        }
        return arr[top];
    }
};

int evaluatePostfix(char exp[]) {
    Stack st;
    int len = strlen(exp);

    for (int i = 0; i < len; i++) {
        char ch = exp[i];

        // Skip spaces
        if (ch == ' ')
            continue;

        // If operand, push it
        if (isdigit(ch)) {
            st.push(ch - '0');  // convert char to int
        }
        else {
            // Operator case
            int val2 = st.pop();
            int val1 = st.pop();

            switch (ch) {
                case '+': st.push(val1 + val2); break;
                case '-': st.push(val1 - val2); break;
                case '*': st.push(val1 * val2); break;
                case '/': st.push(val1 / val2); break;
            }
        }
    }
    return st.pop();
}

int main() {
    char exp[MAX];
    cout << "Enter a postfix expression (single-digit operands): ";
    cin.getline(exp, MAX);
```

```
    cout << "Result = " << evaluatePostfix(exp) << endl;
    return 0;
}
```

Output:

Name: Dishita Bansal Roll No. 1024030365 Batch:2C24

Assignment-4

Ques1. Develop a menu driven program demonstrating the following operations on simple Queues: enqueue(), dequeue(), isEmpty(), isFull(), display(), and peek ().

Soln:

```
#include <iostream>
using namespace std;
#define SIZE 5

class Queue {
private:
    int arr[SIZE];
    int front, rear;

public:
    Queue() {
        front = -1;
        rear = -1;
    }

    bool isEmpty() {
        return (front == -1 && rear == -1);
    }

    bool isFull() {
        return (rear == SIZE - 1);
    }

    void enqueue(int value) {
```

```
    if (isFull()) {
        cout << "Queue Overflow! Cannot insert " << value << endl;
        return;
    }
    if (isEmpty()) {
        front = rear = 0;
    } else {
        rear++;
    }
    arr[rear] = value;
    cout << value << " enqueued into the queue." << endl;
}

void dequeue() {
    if (isEmpty()) {
        cout << "Queue Underflow! Cannot dequeue." << endl;
        return;
    }
    cout << arr[front] << " dequeued from the queue." << endl;
    if (front == rear) {
        front = rear = -1;
    } else {
        front++;
    }
}

void peek() {
    if (isEmpty()) {
```

```
            cout << "Queue is Empty!" << endl;
        } else {
            cout << "Front element is: " << arr[front] << endl;
        }
    }

    void display() {
        if (isEmpty()) {
            cout << "Queue is Empty!" << endl;
            return;
        }
        cout << "Queue elements: ";
        for (int i = front; i <= rear; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
};

int main() {
    Queue q;
    int choice, value;

    do {
        cout << "\n========= Queue Menu ========" << endl;
        cout << "1. Enqueue" << endl;
        cout << "2. Dequeue" << endl;
        cout << "3. Peek" << endl;
```

```
        cout << "4. Display" << endl;
        cout << "5. Check if Empty" << endl;
        cout << "6. Check if Full" << endl;
        cout << "0. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
        case 1:
            cout << "Enter value to enqueue: ";
            cin >> value;
            q.enqueue(value);
            break;
        case 2:
            q.dequeue();
            break;
        case 3:
            q.peek();
            break;
        case 4:
            q.display();
            break;
        case 5:
            if (q.isEmpty())
                cout << "Queue is Empty." << endl;
            else
                cout << "Queue is NOT Empty." << endl;
            break;
```

```
        case 6:
            if (q.isFull())
                cout << "Queue is Full." << endl;
            else
                cout << "Queue is NOT Full." << endl;
            break;
        case 0:
            cout << "Exiting program..." << endl;
            break;
        default:
            cout << "Invalid choice! Try again." << endl;
        }
    } while (choice != 0);

    return 0;
}
```

Output:

```
5. Check if Empty
6. Check if Full
0. Exit
Enter your choice: 3
Front element is: 3

========= Queue Menu ========
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Check if Empty
6. Check if Full
0. Exit
Enter your choice: 4
Queue elements: 3 5 7 10

========= Queue Menu ========
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Check if Empty
6. Check if Full
0. Exit
Enter your choice: 5
Queue is NOT Empty.

========= Queue Menu ========
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Check if Empty
6. Check if Full
0. Exit
Enter your choice: 6
Queue is Full.
```

Ques2. Develop a menu driven program demonstrating the following operations on Circular Queues: enqueue(), dequeue(), isEmpty(), isFull(), display(), and peek().

Soln:
```
#include <iostream>
using namespace std;
#define SIZE 5

class CircularQueue {
private:
    int arr[SIZE];
    int front, rear;

public:
    CircularQueue() {
        front = -1;
        rear = -1;
    }

    bool isEmpty() {
        return (front == -1 && rear == -1);
    }

    bool isFull() {
        return ((rear + 1) % SIZE == front);
    }

    void enqueue(int value) {
        if (isFull()) {
            cout << "Queue Overflow! Cannot insert " << value << endl;
            return;
        }
```

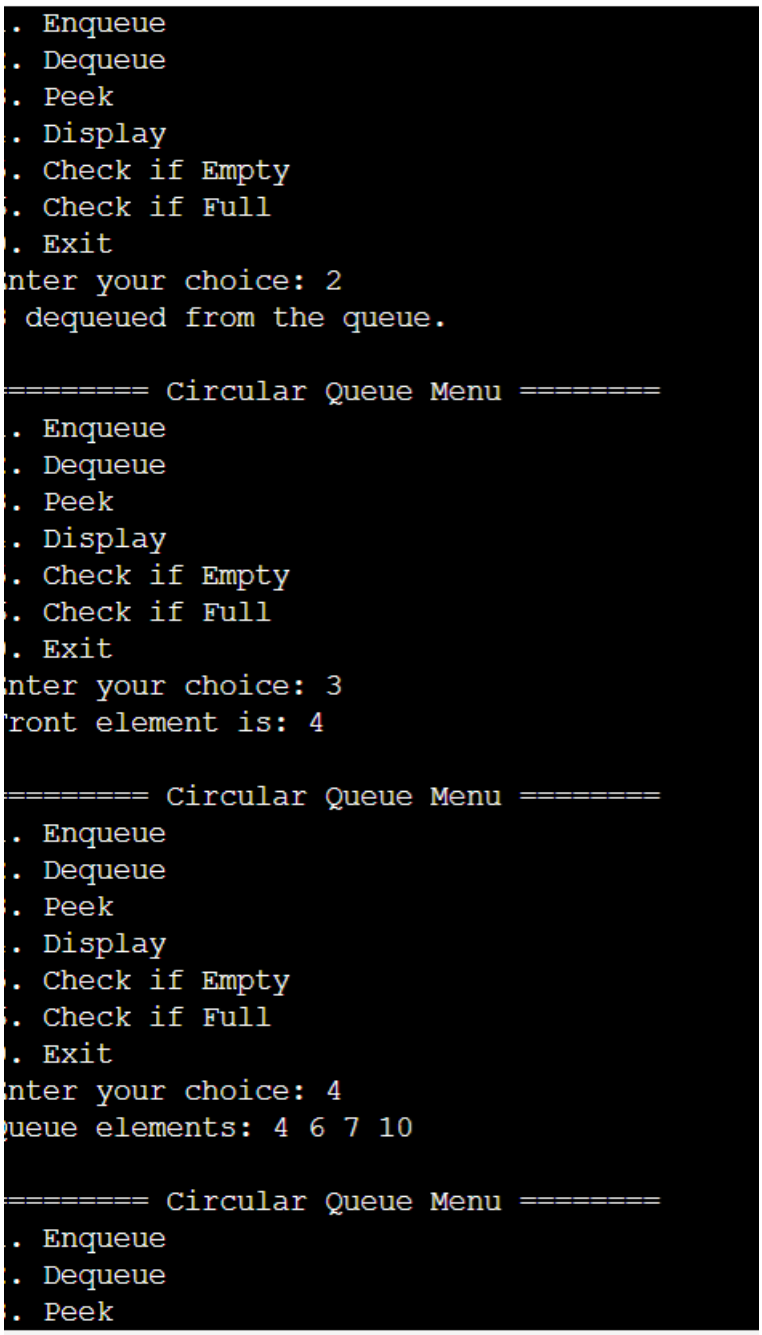
```
    if (isEmpty()) {
        front = rear = 0;
    } else {
        rear = (rear + 1) % SIZE;
    }
    arr[rear] = value;
    cout << value << " enqueued into the queue." << endl;
}

void dequeue() {
    if (isEmpty()) {
        cout << "Queue Underflow! Cannot dequeue." << endl;
        return;
    }
    cout << arr[front] << " dequeued from the queue." << endl;
    if (front == rear) {
        front = rear = -1;
    } else {
        front = (front + 1) % SIZE;
    }
}

void peek() {
    if (isEmpty()) {
        cout << "Queue is Empty!" << endl;
    } else {
        cout << "Front element is: " << arr[front] << endl;
    }
```

```
    }

    void display() {
        if (isEmpty()) {
            cout << "Queue is Empty!" << endl;
            return;
        }
        cout << "Queue elements: ";
        int i = front;
        while (true) {
            cout << arr[i] << " ";
            if (i == rear) break;
            i = (i + 1) % SIZE;
        }
        cout << endl;
    }
};

int main() {
    CircularQueue q;
    int choice, value;

    do {
        cout << "\n========= Circular Queue Menu ========" << endl;
        cout << "1. Enqueue" << endl;
        cout << "2. Dequeue" << endl;
        cout << "3. Peek" << endl;
        cout << "4. Display" << endl;
```

```
        cout << "5. Check if Empty" << endl;
        cout << "6. Check if Full" << endl;
        cout << "0. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
        case 1:
            cout << "Enter value to enqueue: ";
            cin >> value;
            q.enqueue(value);
            break;
        case 2:
            q.dequeue();
            break;
        case 3:
            q.peek();
            break;
        case 4:
            q.display();
            break;
        case 5:
            if (q.isEmpty())
                cout << "Queue is Empty." << endl;
            else
                cout << "Queue is NOT Empty." << endl;
            break;
        case 6:
```

```
            if (q.isFull())
                cout << "Queue is Full." << endl;
            else
                cout << "Queue is NOT Full." << endl;
            break;
        case 0:
            cout << "Exiting program..." << endl;
            break;
        default:
            cout << "Invalid choice! Try again." << endl;
        }
    } while (choice != 0);

    return 0;
}
```

Output:



Ques3. Write a program interleave the first half of the queue with second half.

Sample I/P: 4 7 11 20 5 9 Sample O/P: 4 20 7 5 11 9

Soln : #include <iostream>

using namespace std;

#define MAX 100

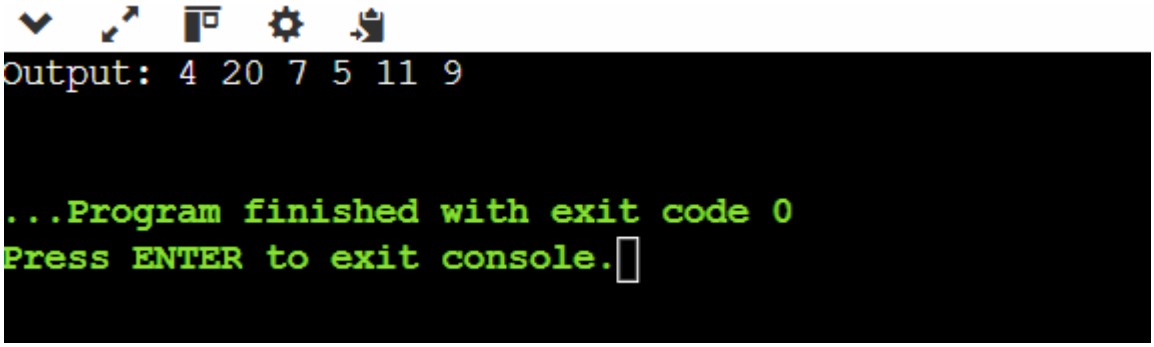
```
class Queue {
    int arr[MAX];
    int front, rear, size;

public:
    Queue() {
        front = 0;
        rear = -1;
        size = 0;
    }

    void enqueue(int x) {
        if (size == MAX) {
            cout << "Queue Overflow\n";
            return;
        }
        rear = (rear + 1) % MAX;
        arr[rear] = x;
        size++;
    }

    int dequeue() {
        if (size == 0) {
            cout << "Queue Underflow\n";
            return -1;
        }
        int val = arr[front];
        front = (front + 1) % MAX;
```

```
        size--;
        return val;
    }

    bool isEmpty() {
        return size == 0;
    }

    int getSize() {
        return size;
    }

    int peek() {
        if (size == 0) return -1;
        return arr[front];
    }
};


void interleaveQueue(Queue &q) {
    int n = q.getSize();
    if (n % 2 != 0) {
        cout << "Queue size must be even!" << endl;
        return;
    }

    int half = n / 2;
    Queue firstHalf;
```

```
    for (int i = 0; i < half; i++) {
        firstHalf.enqueue(q.dequeue());
    }

    // Interleave
    while (!firstHalf.isEmpty()) {
        q.enqueue(firstHalf.dequeue());
        q.enqueue(q.dequeue());
    }
}

int main() {
    Queue q;


    q.enqueue(4);
    q.enqueue(7);
    q.enqueue(11);
    q.enqueue(20);
    q.enqueue(5);
    q.enqueue(9);

    interleaveQueue(q);

    cout << "Output: ";
    while (!q.isEmpty()) {
```

```
        cout << q.dequeue() << " ";
    }
    cout << endl;

    return 0;
}
```

Output:



Ques4. Write a program to find first non-repeating character in a string using Queue. Sample I/P: a a b c Sample O/P: a -1 b b

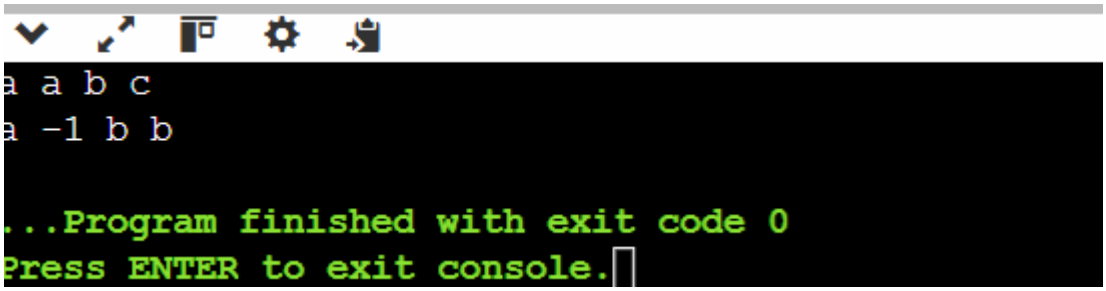
Soln : #include <iostream>

```
using namespace std;

class Queue {
    char arr[100];
    int front, rear;
public:
    Queue() {
        front = 0;
        rear = -1;
    }

    void enqueue(char x) {
        if (rear < 99) {
```

```
            arr[++rear] = x;
        }
    }

    void dequeue() {
      if (!empty()) {
            front++;
        }
    }

    char getFront() {
        return arr[front];
    }

    bool empty() {
        return front > rear;
    }
};

void firstNonRepeating(string str) {
    int freq[256] = {0};
    Queue q;

    for (int i = 0; i < str.length(); i++) {
        char ch = str[i];

        freq[ch]++;
        q.enqueue(ch);
```

```
        while (!q.empty() && freq[q.getFront()] > 1) {
            q.dequeue();
        }


        if (q.empty()) {
            cout << "-1 ";
        } else {
            cout << q.getFront() << " ";
        }
    }
}

int main() {
    string input;
    getline(cin, input);

    string filtered = "";
    for (int i = 0; i < input.length(); i++) {
        if (input[i] != ' ') filtered += input[i];
    }

    firstNonRepeating(filtered);
    return 0;
}
```

Output:



Ques5. Write a program to implement a stack using (a) Two queues and (b) One Queue.

Soln:

```
#include <iostream>
using namespace std;
#define SIZE 100
class Queue {
    int arr[SIZE];
    int front, rear;
public:
    Queue() {
        front = rear = -1;
    }

    bool isEmpty() {
        return (front == -1);
    }

    bool isFull() {
        return (rear == SIZE - 1);
    }

    void enqueue(int x) {
        if (isFull()) {
            cout << "Queue Overflow\n";
```

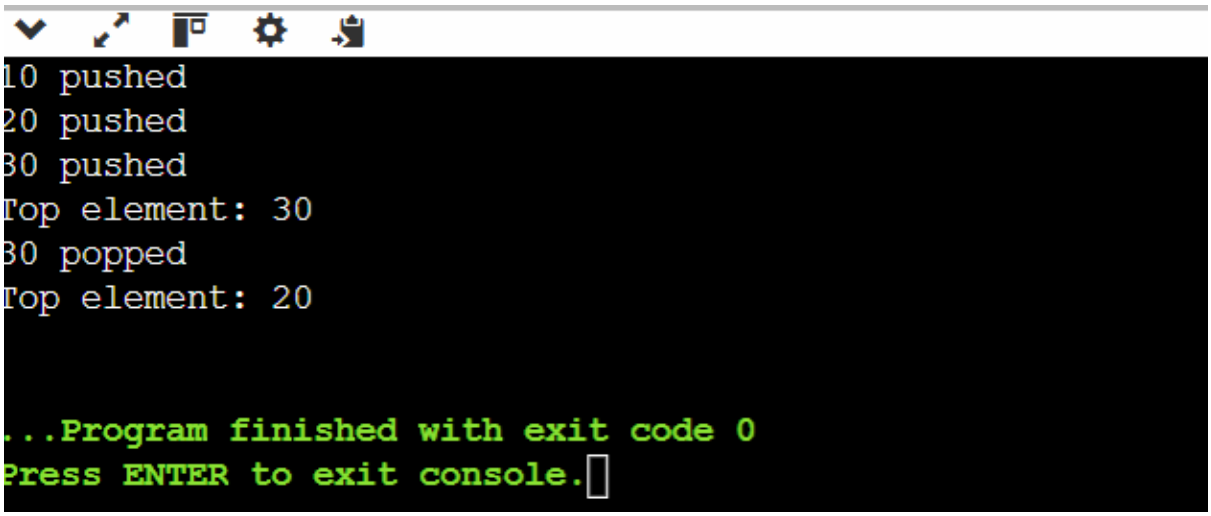
```
        return;
    }
    if (front == -1) front = 0;
    arr[++rear] = x;
}

int dequeue() {
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return -1;
    }
    int val = arr[front];
    if (front == rear)
        front = rear = -1;
    else
        front++;
    return val;
}

int peek() {
    if (isEmpty()) return -1;
    return arr[front];
}

int size() {
    if (isEmpty()) return 0;
    return rear - front + 1;
}
```

```
};

// Stack using Two Queues
class StackTwoQ {
   Queue q1, q2;
public:
   void push(int x) {
      q1.enqueue(x);
      cout << x << " pushed\n";
   }

   void pop() {
      if (q1.isEmpty()) {
         cout << "Stack Underflow\n";
         return;
      }
      while (q1.size() > 1) {
         q2.enqueue(q1.dequeue());
      }
      cout << q1.dequeue() << " popped\n";
      // swap q1 and q2
      Queue temp = q1;
      q1 = q2;
      q2 = temp;
   }

   void top() {
      if (q1.isEmpty()) {
```

```
            cout << "Stack is Empty\n";
            return;
        }
        while (q1.size() > 1) {
            q2.enqueue(q1.dequeue());
        }
        int val = q1.dequeue();
        cout << "Top element: " << val << endl;
        q2.enqueue(val);
        Queue temp = q1;
        q1 = q2;
        q2 = temp;
    }

    bool empty() {
        return q1.isEmpty();
    }
};

int main() {
    StackTwoQ s;
    s.push(10);
    s.push(20);
    s.push(30);
    s.top();
    s.pop();
    s.top();
    return 0;
```

}

Output:


10 pushed
20 pushed
30 pushed
Top element: 30
30 popped
Top element: 20
...Program finished with exit code 0
Press ENTER to exit console.

Name: Dishita Bansal Roll No. 1024030365 Batch:2C24

ASSIGNMENT -5

QUES 1: 1. Develop a menu driven program for the following operations on a Singly Linked List. (a) Insertion at the beginning. (b) Insertion at the end. (c) Insertion in between (before or after a node having a specific value, say 'Insert a new Node 35 before/after the Node 30'). (d) Deletion from the beginning. (e) Deletion from the end. (f) Deletion of a specific node, say 'Delete Node 60'). (g) Search for a node and display its position from head. (h) Display all the node values.

SOLUTION:
```
#include<iostream>

using namespace std;
class node{
    public:
    int data;
    node* next;
    node(int value){
        data = value;
        next = nullptr;
    }
};
class linkedlist{
    private:
    node* head;
    public:
    linkedlist(){
        head = nullptr;
    }
    void insertatbegining(int value){
        node* newnode = new node(value);
        newnode->next = head;
        head = newnode;
    }
    void insertatend(int value){
        node* newnode = new node(value);
        if(head == nullptr){
            head = newnode;
            return;
        }
        node* temp = head;
        while(temp->next!=nullptr){
            temp = temp->next;
        }
        temp->next = newnode;
    }
    void insertafter(int prevalue,int value){
        node* temp = head;
    while (temp != nullptr && temp->data != prevalue) {
```

```
        temp = temp->next;
    }
    if (temp == nullptr) {
        cout << "Previous node not found!\n";
        return;
    }
    node* newNode = new node(value);
    newNode->next = temp->next;
    temp->next = newNode;
    }
    void deleteatbeginning(){
        if (head == nullptr){
            return;
        }
        node* temp = head;
        head = head->next;
        delete temp;
    }
    void deleteatend(){
        if(head == nullptr){
            return;
        }
        if(head->next == nullptr){
            delete head;
            head = nullptr;
            return;
        }
        node* temp = head;
        while(temp->next->next!= nullptr){
            temp = temp->next;
        }
        delete temp->next;
        temp->next = nullptr;
    }
    void deletebyvalue(int value){
        if(head == nullptr){
            return;
        }
        if(head->data == value){
            node* temp = head;
            head = head->next;
            delete temp;
            return;
        }
        node* temp = head;
        node* prevnode = nullptr;
        while(temp!=nullptr && temp->data!= value){
            prevnode = temp;
```

```
            temp = temp->next;
        }
        if(temp == nullptr){
            return;
        }
        prevnode->next = temp->next;
        delete temp;

    }
    int search(int key) {
        node* temp = head;
        int pos = 1;

        while (temp != nullptr) {
            if (temp->data == key) {
                return pos;
                cout<<"value found at "<<pos;
            }
            temp = temp->next;
            pos++;
        }
        return -1;
    }
    void printlist(){
        node* temp = head;
        while(temp!=nullptr){
            cout<<temp->data<<" -> ";
            temp = temp->next;
        }
    }

};
int main(){
    linkedlist l;
    int choice;
    do{
    cout<<"\n==MENU DRIVEN==\n";
    cout<<"Option 1: Insertion at the beginning\n";
    cout<<"Option 2: Insertion at the end\n";
    cout<<"Option 3: Insertion in between\n";
    cout<<"Option 4: Deletion from the beginning.\n";
    cout<<"Option 5: Deletion from the end.\n";
    cout<<"Option 6: Deletion of a specific node\n";
    cout<<"Option 7: Search in linked list .\n";
    cout<<"Option 8: Display all the node values.\n";
    cout<<"Option 9: Exit\n";
    cout<<"Enter your choice:";
    cin>>choice;
```

```
    switch(choice){
        case 1 :
        int value;
        cout<<"Value: ";
        cin>>value;
        l.insertatbegining(value);
        break;
        case 2 :
        int value1;
        cout<<"Value: ";
        cin>>value1;
        l.insertatend(value1);
        break;
        case 3 :
        int value5,value2;
        cout<<"Value: ";
        cin>>value5;
        cout<<" prev node value: ";
        cin>>value2;
        l.insertafter(value2,value5);
        break;
        case 4 :
        l.deleteatbeginning();
        break;
        case 5:
        l.deleteatend();
        break;
        case 6:
        int value4;
        cout<<"Value to be deleted";
        cin>>value4;
        l.deletebyvalue(value4);
        break;
        case 7:
        int value3;
        cout<<"Value to be searched :";
        cin>>value3;
        l.search(value3);
        break;
        case 8 :
        l.printlist();
        break;
        case 9:
        cout<<"Exit";
    }
    }while(choice!=9);


}
```

Name: Dishita Bansal Roll No. 1024030365 Batch:2C24

Output:

```
==MENU DRIVEN==
Option 1: Insertion at the beginning
Option 2: Insertion at the end
Option 3: Insertion in between
Option 4: Deletion from the beginning.
Option 5: Deletion from the end.
Option 6: Deletion of a specific node
Option 7: Search in linked list .
Option 8: Display all the node values.
Option 9: Exit
Enter your choice:1
Value: 10

==MENU DRIVEN==
Option 1: Insertion at the beginning
Option 2: Insertion at the end
Option 3: Insertion in between
Option 4: Deletion from the beginning.
Option 5: Deletion from the end.
Option 6: Deletion of a specific node
Option 7: Search in linked list .
Option 8: Display all the node values.
Option 9: Exit
```

Ln 67, Col 43 Spaces: 4 UTF-8 CRLF {} C++

```
 Option 2: Insertion at the end
 Option 3: Insertion in between
 Option 4: Deletion from the beginning.
 Option 5: Deletion from the end.
 Option 6: Deletion of a specific node
 Option 7: Search in linked list .
 Option 8: Display all the node values.
 Option 9: Exit
 Enter your choice:2
 Value: 40

 ==MENU DRIVEN==
 Option 1: Insertion at the beginning
 Option 2: Insertion at the end
 Option 3: Insertion in between
 Option 4: Deletion from the beginning.
 Option 5: Deletion from the end.
 Option 6: Deletion of a specific node
 Option 7: Search in linked list .
 Option 8: Display all the node values.
 Option 9: Exit
 Enter your choice:
 8
 10 -> 20 -> 30 -> 40 ->
 ==MENU DRIVEN==
```

```
Enter your choice:4

==MENU DRIVEN==
Option 1: Insertion at the beginning
Option 2: Insertion at the end
Option 3: Insertion in between
Option 4: Deletion from the beginning.
Option 5: Deletion from the end.
Option 6: Deletion of a specific node
Option 7: Search in linked list .
Option 8: Display all the node values.
Option 9: Exit
Enter your choice:8
20 -> 30 -> 40 ->
```

```
Enter your choice:5

==MENU DRIVEN==
Option 1: Insertion at the beginning
Option 2: Insertion at the end
Option 3: Insertion in between
Option 4: Deletion from the beginning.
Option 5: Deletion from the end.
Option 6: Deletion of a specific node
Option 7: Search in linked list .
Option 8: Display all the node values.
Option 9: Exit
Enter your choice:8
20 -> 30 ->
Enter your choice:9
Exit
```

Ques 2:Write a program to count the number of occurrences of a given key in a singly linked list and then delete all the occurrences. Input: Linked List : 1->2->1->2->1->3->1 , key: 1 Output: Count: 4 , Updated Linked List: 2->2->3.

Solution:

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) {
        data = val;
        next = NULL;
    }
};

// Function to insert node at end
void insert(Node*& head, int val) {
    Node* newNode = new Node(val);
    if (head == NULL) {
        head = newNode;
        return;
    }
    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}

// Function to display linked list
void display(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data;
```

```
        if (temp->next != NULL) cout << "->";
        temp = temp->next;
    }
    cout << endl;
}

// Function to count occurrences and delete them
int countAndDelete(Node*& head, int key) {
    int count = 0;

    // Handle head nodes having the key
    while (head != NULL && head->data == key) {
        Node* toDelete = head;
        head = head->next;
        delete toDelete;
        count++;
    }

    Node* current = head;
    Node* prev = NULL;

    while (current != NULL) {
        if (current->data == key) {
            count++;
            prev->next = current->next;
            delete current;
            current = prev->next;
        } else {
            prev = current;
            current = current->next;
        }
    }

    return count;
}

// Driver Code
int main() {
    Node* head = NULL;

    // Creating linked list: 1->2->1->2->1->3->1
    insert(head, 1);
    insert(head, 2);
    insert(head, 1);
    insert(head, 2);
    insert(head, 1);
    insert(head, 3);
    insert(head, 1);
```

```
    cout << "Original Linked List: ";
    display(head);

    int key = 1;
    int count = countAndDelete(head, key);

    cout << "Count of " << key << ": " << count << endl;
    cout << "Updated Linked List: ";
    display(head);

    return 0;
}
```

Output:

```
Original Linked List: 1->2->1->2->1->3->1
Count of 1: 4
Updated Linked List: 2->2->3
```

Ques 3 : 3. Write a program to find the middle of a linked list. Input: 1->2->3->4->5
Output: 3

Solution:

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(NULL) {}
};

// Insert at end
void insert(Node*& head, int val) {
    Node* newNode = new Node(val);
    if (!head) {
        head = newNode;
        return;
    }
    Node* temp = head;
    while (temp->next) temp = temp->next;
    temp->next = newNode;
}
```

Name: Dishita Bansal Roll No. 1024030365 Batch:2C24

```
// Find middle node
int findMiddle(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while (fast && fast->next) {
        slow = slow->next;       // move 1 step
        fast = fast->next->next; // move 2 steps
    }
    return slow->data;
}

// Driver code
int main() {
    Node* head = NULL;

    // Linked List: 1->2->3->4->5
    insert(head, 1);
    insert(head, 2);
    insert(head, 3);
    insert(head, 4);
    insert(head, 5);

    cout << "Middle element: " << findMiddle(head) << endl;
    return 0;
}
s
```

solution:

```
Middle element: 3
```

Name: Dishita Bansal Roll No.1024030365 Batch:2C24

DSA ASSIGNMENT-6

Ques 1 :

Solution:

```
#include<iostream>
using namespace std;
class node{
    public:
    int data;
    node* next;
    node* prev;
    node(int num){
        data = num;
        next = nullptr;
        prev = nullptr;
    }
};
class linkedlist{
    public:
    node* head;
    node* tail;
    linkedlist(){
        head = nullptr;
        tail = nullptr;
    }
    void insertatbeginning(int data){
        node* newnode = new node(data);
        if(head == nullptr){
            head = tail = newnode;
            return;
        }
        newnode->next = head;
        head->prev = newnode;
        head = newnode;
    }
    void insertatend(int data){
        node* newnode = new node(data);
        if (tail == nullptr){
            head = tail = newnode;
            return;
        }
        tail->next = newnode;
        tail->next->prev = tail;
        tail = newnode;
     }
     void insert_after(int value,int pos){
```

```
        node* newnode = new node(pos);
        if(head == nullptr){
            cout<<"List empty";
            return;
        }
        node* temp = head;
        int i = 1;
        while(i<pos && temp->next!=nullptr){    //for insert before just press
i = pos-1 ;
            temp = temp->next;
            i++;
        }
        if(temp==tail){
            temp->next = newnode;
            newnode->prev = temp;
            tail = newnode;
        }
        else{
            newnode->next = temp->next;
            newnode->prev=temp;
            temp->next->prev = newnode;
            temp->next = newnode;
        }
    }
    void delete_beginning(){
        node* temp = head;
        if(!head){
            cout<<"List empty";
        }
        if(head ==tail){
            head = tail = nullptr;
        }
        else{
            head = head->next;
            head->prev = nullptr;
        }
        delete temp;
        cout<<"First node deleted";
    }
    void delete_at_end(){
        node* temp = tail;
        if(!tail){
            cout<<"List empty";
        }
        if(head == tail){
            head = tail = nullptr;
        }
        else{
```

```
            tail = tail->prev;
            tail->next = nullptr;
        }
        delete temp;
        cout<<"Last node deleted";
    }
    void delete_at_specific(int pos){
        node* temp = head;
        int i = 1 ;
        if(!head){
            cout<<"List empty";
        }
        if(pos<0){cout<<"Invalid pos";}
        if(pos ==1){delete_beginning();}
        while(i<pos && temp->next!= nullptr){
            temp = temp->next;
            i++;
        }
        if(temp==tail){delete_at_end();}
        temp->prev->next = temp->next;
        temp->next->prev = temp->next;

    }
    void display_forward(){
        if(!head){
            cout<<"List empty";

        }
        cout<<"Forward";
        node* temp = head;
        while(temp != nullptr){
            cout<<temp->data<<"->";
            temp = temp->next;
        }
    }
    void display_backward(){
        node* temp = tail;
        cout<<"backward display";
        if(!head){
            cout<<"List empty";
        }
        while(temp){
            cout<<temp->data;
            cout<<endl;
            temp = temp->prev;
        }
    }};
    main(){
```

```
    linkedlist ll;
    int choice,value,pos;
do {
    cout << "\n===== DOUBLY LINKED LIST MENU =====\n";
    cout << "1. Insert at Beginning\n";
    cout << "2. Insert at End\n";
    cout << "3. Insert After Position\n";
    cout << "4. Delete Beginning\n";
    cout << "5. Delete End\n";
    cout << "6. Delete at Specific Position\n";
    cout << "7. Display Forward\n";
    cout << "8. Display Backward\n";
    cout << "9. Exit\n";
    cout << "Enter your choice: ";
    cin >> choice;
    switch (choice) {
        case 1:
            cout << "Enter value: ";
            cin >> value;
            ll.insertatbeginning(value);
            break;
        case 2:
            cout << "Enter value: ";
            cin >> value;
            ll.insertatend(value);
            break;
        case 3:
            cout << "Enter position and value: "; cin >> pos >> value;
            ll.insert_after(value,pos);
            break;
         case 4:
            ll.delete_beginning();
            break;
        case 5:
            ll.delete_at_end();
            break;
        case 6:
            cout << "Enter position to delete: "; cin >> pos;
            ll.delete_at_specific(pos);
            break;
        case 7:
            ll.display_forward();
            break;
        case 8:
            ll.display_backward();
            break;
        case 9:
            cout << "Exiting...\n";
```

```
                break;
            default:
                cout << "Invalid choice!\n";


    }}while(choice!=9);
}
```

Ques 2:

Solution:

```
#include<iostream>
using namespace std;
class node{
    public:
    int data;
    node* next;
    node(int value){
        data = value;
        next = nullptr;
    }
};
class circularlinkedlist{
    public:
    node* head;
    circularlinkedlist(){
        head = nullptr;
}
void inserrtatbeginning(int value){
    node* newnode = new node(value);
    if(head ==nullptr){
        head = newnode;
        newnode->next = head;
        return;
    }
    node* temp = head;
    while(temp->next!=head){
        temp = temp->next;
    }
    temp->next = newnode;
    newnode->next = head;
    head = newnode;

}
void display(){
    if(head == nullptr){
        cout<<"list empty";
```

```
    }
    node* temp = head;
    while(temp->next != head){
        cout<<temp->data<<" ";
        temp = temp->next;
    }
    cout<<temp->data<<" "<<head->data;
}};
main(){
    circularlinkedlist cll;
    cll.inserrtatbeginning(60);
    cll.inserrtatbeginning(80);
    cll.inserrtatbeginning(40);
    cll.inserrtatbeginning(100);
    cll.inserrtatbeginning(20);
    cll.display();



}
```

Solution:

```
20 100 40 80 60 20
```

Ques3 :

Solution:

```
#include<iostream>
using namespace std;
class node{
    public:
    int data;
    node* next;
    node* prev;
    node(int value){
        data = value;
        next = nullptr;
        prev = nullptr;

    }
};
class doublylinkedlist{
    public:
    node* head;
    doublylinkedlist(){
```

```
        head = nullptr;
    }
    void insertatbeginning(int value){
        node* newnode = new node(value);
        if(head == nullptr){
            head = newnode;
            return;
        }
        newnode->next = head;
        head->prev= newnode;
        head = newnode;
    }
    int size(){
        int count = 0;
        node* temp = head;
        while(temp!=nullptr){
            count++;
            temp = temp->next;
        }
        return count;
    }
};
main(){
    doublylinkedlist dll;
    dll.insertatbeginning(40);
    dll.insertatbeginning(30);
    dll.insertatbeginning(20);
    dll.insertatbeginning(10);
    cout<<"Size of doubly linked list :"<<dll.size();
}
```

Output:

```
o 3 } ; if ($?) { .\3 }
Size of doubly linked list :4
```

b)

```
#include<iostream>
using namespace std;
class node{
    public:
    int data;
    node* next;
    node(int value){
        data = value;
        next = nullptr;
    }
};
class circularlinkedlist{
```

```
    public:
    node* head;
    circularlinkedlist(){
        head = nullptr;
    }
    void insertatbeginning(int value)
    {
        node* newnode = new node(value);
        if(head == nullptr){
            head= newnode;
            newnode->next = head;
            return;
        }
        node* temp = head;
        while(temp->next!=head){
            temp = temp->next;
        }
        temp->next = newnode;
        newnode->next = head;
        head = newnode;
    }
    int size(){
        int count = 0;
        node* temp = head;
        while(temp->next= head){
            count++;
            temp = temp->next;

        }
        return count;
    }
};
main(){
    circularlinkedlist cll;
    cll.insertatbeginning(40);
    cll.insertatbeginning(30);
    cll.insertatbeginning(20);
    cll.insertatbeginning(10);
    cout<<"Size of circular linked list :"<<cll.size();
}
```

Output:

```
o 3 } ; if ($?) { .\3 }
Size of doubly linked list :4
```

Ques 4 :

Solution:

```
#include<iostream>
```

Name: Dishita Bansal Roll No.1024030365 Batch:2C24

```
using namespace std;
class node{
    public:
    char data;
    node* next;
    node* prev;
    node(char value){
        data = value;
        next = nullptr;
        prev = nullptr;
    }
};
class linkedlist{
    public:
    node* head;

    linkedlist(){
        head = nullptr;
    }
    void insert_at_end(char value){
        node* newnode = new node(value);
        if(head == nullptr){
            head = newnode;
            return;
        }
        node* temp = head;
        while(temp->next!=nullptr){
            temp = temp->next;
        }
        temp->next=newnode;
        newnode->prev = temp;
    }
    bool is_pallindrome(){
        if(!head || !head->next ){
            return true;
        }
        node* tail = head;
        while(tail->next){
            tail = tail->next;
        }
        while(head!= tail && tail->next!=head){
            if(head->data!=tail->data){
                return false;
            }
            head = head->next;
            tail = tail->prev;
        }
        return true;
```

```
    }

};
main(){
    linkedlist ll;
    ll.insert_at_end('m');
    ll.insert_at_end('a');
    ll.insert_at_end('d');
    ll.insert_at_end('a');
    ll.insert_at_end('m');
    cout<<"cycle :"<<ll.is_pallindrome();

}
```

Output:

```
cycle :1

...Program finished with exit code 0
Press ENTER to exit console.
```

Ques 5:

Solution:

```
#include<iostream>
using namespace std;
class node{
    public:
    int data;
    node* next;
    node(int value){
        data = value;
        next = nullptr;
    }
};
class linkedlist{
    public:
    node* head;
    linkedlist(){
        head = nullptr;
    }
    void insertatend(int value){
        node* newnode = new node(value);
        if(head == nullptr){
            head = newnode;
            newnode->next = head;
            return;
```

```
        }
        node* temp = head;
        while(temp ->next != head){
            temp = temp->next;
        }
        temp->next=newnode;
        newnode->next = head;
    }
    bool detect_cycle() {
    if (head == nullptr) return false;

    node* slow = head;
    node* fast = head;

    do {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast) return true;
    } while (fast != nullptr && fast->next != nullptr);

    return false;
}

};
int main(){
    linkedlist ll;
    ll.insertatend(10);
    ll.insertatend(20);
    ll.insertatend(30);
    ll.insertatend(40);
    cout<<"cycle is "<<ll.detect_cycle();
    return 0;
}
```

Output:

```
cycle is 1

...Program finished with exit code 0
Press ENTER to exit console.
```

Name: Dishita Bansal Roll No. 1024030365
Batch:2C24

DSA ASSIGNMENT -SORTING

QUES 1: Write a program to implement following sorting techniques:

a. Selection Sort

SOLUTION :
```
#include<iostream>
using namespace std;
main(){
int arr[5]={3,4,2,1,5};
int n = 5;
int min;
for(int i = 0;i<=n-2;i++){
    min = i;
    for(int j=i+1;j<=n-1;j++){
        if(arr[j]<arr[min]){
            min = j;
        }
    }
    if(min!=i){
        int temp = arr[min];
        arr[min]=arr[i];
        arr[i]= temp;
    }

}
for(int i = 0;i<n;i++){
    cout<<arr[i];
    cout<<endl;
}
}
```

OUTPUT:

```
1
2
3
4
5
```

QUES 2: Insertion Sort

SOLUTION:

```
#include <iostream>
using namespace std;
```

Name: Dishita Bansal Roll No. 1024030365
Batch:2C24

```cpp
int main() {
    int n;
    cin >> n;
    int a[n];

    for(int i = 0; i < n; i++)
        cin >> a[i];

    for(int i = 1; i < n; i++) {
        int temp = a[i];
        int j = i;

        while(j > 0 && a[j-1] > temp) {
            a[j] = a[j-1];
            j--;
        }
        a[j] = temp;
    }

    for(int i = 0; i < n; i++)
        cout << a[i] << " ";

    return 0;
}
```

Output:

```
7
1 4 5 3 6 2 7
1 2 3 4 5 6 7
```

Ques 3: Bubble sort ,merge sort,quick sort

```cpp
#include <iostream>
using namespace std;

void mySwap(int &x, int &y) {
    int temp = x;
    x = y;
    y = temp;
}

void bubbleSort(int a[], int n) {
    for(int i = 0; i < n - 1; i++)
        for(int j = 0; j < n - i - 1; j++)
            if(a[j] > a[j + 1])
                mySwap(a[j], a[j + 1]);
}
```

Name: Dishita Bansal Roll No. 1024030365
Batch:2C24

```
void merge(int a[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];

    for(int i = 0; i < n1; i++) L[i] = a[l + i];
    for(int i = 0; i < n2; i++) R[i] = a[m + 1 + i];

    int i = 0, j = 0, k = l;

    while(i < n1 && j < n2)
        a[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];

    while(i < n1) a[k++] = L[i++];
    while(j < n2) a[k++] = R[j++];
}

void mergeSort(int a[], int l, int r) {
    if(l < r) {
        int m = (l + r) / 2;
        mergeSort(a, l, m);
        mergeSort(a, m + 1, r);
        merge(a, l, m, r);
    }
}

int partition(int a[], int low, int high) {
    int pivot = a[high], i = low - 1;

    for(int j = low; j < high; j++)
        if(a[j] < pivot)
            mySwap(a[++i], a[j]);

    mySwap(a[i + 1], a[high]);
    return i + 1;
}

void quickSort(int a[], int low, int high) {
    if(low < high) {
        int pi = partition(a, low, high);
        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}

void printArray(int a[], int n) {
    for(int i = 0; i < n; i++) cout << a[i] << " ";
    cout << endl;
}
```

Name: Dishita Bansal Roll No. 1024030365
Batch:2C24

```
int main() {
    int n;
    cin >> n;
    int a[n];
    cout<<"Unsorted array:";
    for(int i = 0; i < n; i++) cin >> a[i];

    bubbleSort(a, n);
    cout<<"bubble sort:";
    printArray(a, n);

    mergeSort(a, 0, n - 1);
    cout<<"merge sort:";
    printArray(a, n);

    quickSort(a, 0, n - 1);
    cout<<"quick sort:";
    printArray(a, n);

    return 0;
}
```

Output:

```
5
Unsorted array1 2 5 4 3
bubble sort:1 2 3 4 5
merge sort:1 2 3 4 5
quick sort:1 2 3 4 5
```

Name: Dishita Bansal Roll No. 1024030365
Batch:2C24

DSA ASSIGNMENT

QUES 1: Write program using functions for binary tree traversals: Pre-order, In-order and Post order using recursive approach.

SOLUTION:

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node *left, *right;

    Node(int value) {
        data = value;
        left = right = NULL;
    }
};

Node* createNode(int value) {
    return new Node(value);
}

void preorder(Node* root) {
    if (root == NULL) return;
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}

void inorder(Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

void postorder(Node* root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}

int main() {
    Node* root = createNode(1);
```

Name: Dishita Bansal Roll No. 1024030365
Batch:2C24

```
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);

    cout << "Preorder Traversal: ";
    preorder(root);

    cout << "\nInorder Traversal: ";
    inorder(root);

    cout << "\nPostorder Traversal: ";
    postorder(root);

    return 0;
}
```

Output:

```
 if ($?) { .\1 }
Preorder Traversal: 1 2 4 5 3
Inorder Traversal: 4 2 5 1 3
Postorder Traversal: 4 5 2 3 1
```

Ques 2:Implement following functions for Binary Search Trees (a) Search a given item (Recursive & Non-Recursive) (b) Maximum element of the BST (c) Minimum element of the BST (d) In-order successor of a given node the BST (e) In-order predecessor of a given node the BST

Solution:

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node *left, *right;

    Node(int value) {
        data = value;
        left = right = NULL;
    }
};

Node* insertNode(Node* root, int value) {
    if (root == NULL) return new Node(value);
```

Name: Dishita Bansal Roll No. 1024030365
Batch:2C24

```
    if (value < root->data) root->left = insertNode(root->left, value);
    else root->right = insertNode(root->right, value);
    return root;
}

Node* searchRec(Node* root, int key) {
    if (root == NULL || root->data == key) return root;
    if (key < root->data) return searchRec(root->left, key);
    return searchRec(root->right, key);
}

Node* searchNonRec(Node* root, int key) {
    while (root != NULL) {
        if (key == root->data) return root;
        else if (key < root->data) root = root->left;
        else root = root->right;
    }
    return NULL;
}

Node* findMin(Node* root) {
    while (root && root->left != NULL)
        root = root->left;
    return root;
}

Node* findMax(Node* root) {
    while (root && root->right != NULL)
        root = root->right;
    return root;
}

Node* inorderSuccessor(Node* root, Node* target) {
    if (target->right != NULL)
        return findMin(target->right);

    Node* succ = NULL;
    while (root != NULL) {
        if (target->data < root->data) {
            succ = root;
            root = root->left;
        }
        else if (target->data > root->data)
            root = root->right;
        else
            break;
    }
    return succ;
```

```
}

Node* inorderPredecessor(Node* root, Node* target) {
    if (target->left != NULL)
        return findMax(target->left);

    Node* pred = NULL;
    while (root != NULL) {
        if (target->data > root->data) {
            pred = root;
            root = root->right;
        }
        else if (target->data < root->data)
            root = root->left;
        else
            break;
    }
    return pred;
}

int main() {
    Node* root = NULL;
    int arr[] = {50, 30, 20, 40, 70, 60, 80};
    for (int i = 0; i < 7; i++)
        root = insertNode(root, arr[i]);

    int key = 40;

    Node* r = searchRec(root, key);
    if (r) cout << "Found (Recursive): " << r->data << endl;
    else cout << "Not Found (Recursive)" << endl;

    Node* nr = searchNonRec(root, key);
    if (nr) cout << "Found (Non-Recursive): " << nr->data << endl;
    else cout << "Not Found (Non-Recursive)" << endl;

    Node* mn = findMin(root);
    cout << "Min: " << mn->data << endl;

    Node* mx = findMax(root);
    cout << "Max: " << mx->data << endl;

    Node* suc = inorderSuccessor(root, nr);
    if (suc) cout << "Successor of " << key << ": " << suc->data << endl;
    else cout << "No Successor" << endl;

    Node* pre = inorderPredecessor(root, nr);
    if (pre) cout << "Predecessor of " << key << ": " << pre->data << endl;
```

```
    else cout << "No Predecessor" << endl;

    return 0;
}
```

Output:

```
Found (Recursive): 40
Found (Non-Recursive): 40
Min: 20
Max: 80
Successor of 40: 50
Predecessor of 40: 30
```

Ques 3:Write a program for binary search tree (BST) having functions for the following operations: (a) Insert an element (no duplicates are allowed), (b) Delete an existing element, (c) Maximum depth of BST (d) Minimum depth of

Solution:

```
#include <iostream>

using namespace std;

class Node {
public:
    int data;
    Node *left, *right;
    Node(int v) { data = v; left = right = NULL; }
};

Node* insertNode(Node* r, int v) {
    if (!r) return new Node(v);
    if (v < r->data) r->left = insertNode(r->left, v);
    else if (v > r->data) r->right = insertNode(r->right, v);
    return r;
}

Node* minNode(Node* r) {
    while (r->left) r = r->left;
    return r;
}

Node* deleteNode(Node* r, int k) {
    if (!r) return r;
    if (k < r->data) r->left = deleteNode(r->left, k);
    else if (k > r->data) r->right = deleteNode(r->right, k);
    else {
```

Name: Dishita Bansal Roll No. 1024030365
Batch:2C24

```
        if (!r->left) { Node* t = r->right; delete r; return t; }
        if (!r->right) { Node* t = r->left; delete r; return t; }
        Node* t = minNode(r->right);
        r->data = t->data;
        r->right = deleteNode(r->right, t->data);
    }
    return r;
}

int maxDepth(Node* r) {
    if (!r) return 0;
    int l = maxDepth(r->left), h = maxDepth(r->right);
    return 1 + (l > h ? l : h);
}

int minDepth(Node* r) {
    if (!r) return 0;
    int l = minDepth(r->left), h = minDepth(r->right);
    if (!r->left) return 1 + h;
    if (!r->right) return 1 + l;
    return 1 + (l < h ? l : h);
}

void inorder(Node* r) {
    if (!r) return;
    inorder(r->left);
    cout << r->data << " ";
    inorder(r->right);
}

int main() {
    Node* root = NULL;
    int a[] = {50, 30, 20, 40, 70, 60, 80};
    for (int i = 0; i < 7; i++) root = insertNode(root, a[i]);

    inorder(root);
    root = deleteNode(root, 40);
}
```

Output:

```
f ($?) { .\3 }
20 30 40 50 60 70 80
```

Ques 4: Write a program to determine whether a given binary tree is a BST or not

```
#include<iostream>
using namespace std;
```

Name: Dishita Bansal Roll No. 1024030365
Batch:2C24

```cpp
class Node {
public:
    int data;
    Node *left, *right;
    Node(int v) { data = v; left = right = NULL; }
};

Node* newNode(int v) {
    return new Node(v);
}

bool isBSTUtil(Node* root, int minV, int maxV) {
    if (root == NULL) return true;
    if (root->data <= minV || root->data >= maxV) return false;
    return isBSTUtil(root->left, minV, root->data) &&
           isBSTUtil(root->right, root->data, maxV);
}

bool isBST(Node* root) {
    return isBSTUtil(root, -1000000, 1000000);
}

int main() {
    Node* root = newNode(4);
    root->left = newNode(2);
    root->right = newNode(6);
    root->left->left = newNode(1);
    root->left->right = newNode(3);

    if (isBST(root)) cout << "Tree is a BST";
    else cout << "Tree is NOT a BST";

    return 0;
}
```

Output:

```
Tree is a BST
```

Ques 5:

```cpp
#include <iostream>
using namespace std;

void maxHeapify(int a[], int n, int i) {
    int largest = i;
    int l = 2*i + 1, r = 2*i + 2;
```

Name: Dishita Bansal Roll No. 1024030365
Batch:2C24

```
    if (l < n && a[l] > a[largest]) largest = l;
    if (r < n && a[r] > a[largest]) largest = r;
    if (largest != i) {
        int t = a[i]; a[i] = a[largest]; a[largest] = t;
        maxHeapify(a, n, largest);
    }
}

void minHeapify(int a[], int n, int i) {
    int smallest = i;
    int l = 2*i + 1, r = 2*i + 2;
    if (l < n && a[l] < a[smallest]) smallest = l;
    if (r < n && a[r] < a[smallest]) smallest = r;
    if (smallest != i) {
        int t = a[i]; a[i] = a[smallest]; a[smallest] = t;
        minHeapify(a, n, smallest);
    }
}

void heapSortIncreasing(int a[], int n) {
    for (int i = n/2 - 1; i >= 0; i--)
        maxHeapify(a, n, i);
    for (int i = n-1; i > 0; i--) {
        int t = a[0]; a[0] = a[i]; a[i] = t;
        maxHeapify(a, i, 0);
    }
}

void heapSortDecreasing(int a[], int n) {
    for (int i = n/2 - 1; i >= 0; i--)
        minHeapify(a, n, i);
    for (int i = n-1; i > 0; i--) {
        int t = a[0]; a[0] = a[i]; a[i] = t;
        minHeapify(a, i, 0);
    }
}

int main() {
    int a[] = {12, 11, 13, 5, 6, 7};
    int n = 6;

    heapSortIncreasing(a, n);
    cout << "Increasing order: ";
    for (int i = 0; i < n; i++) cout << a[i] << " ";

    int b[] = {12, 11, 13, 5, 6, 7};
    heapSortDecreasing(b, n);
    cout << "\nDecreasing order: ";
```

Name: Dishita Bansal Roll No. 1024030365
Batch:2C24

```
    for (int i = 0; i < n; i++) cout << b[i] << " ";

    return 0;
}
```

Output:

```
Increasing order: 5 6 7 11 12 13
Decreasing order: 13 12 11 7 6 5
```

Name: Dishita Bansal Roll No.1024030365
Batch:2C24

DSA ASSIGNMENT – 9

QUES 1: A graph G is defined as a pair (V, E) where V is a set of nodes/vertices and E is a set of edges connecting pairs of vertices. Graphs may be directed or undirected and may have weighted or unweighted edges. They can be represented using an adjacency matrix, adjacency list, or edge list. Write a program to implement the following graph algorithms: 1. Breadth First Search (BFS) 2. Depth First Search (DFS) 3. Minimum Spanning Tree (Kruskal and Prim) 4. Dijkstra's Shortest Path Algorithm

SOLUTION:

```
#include <iostream>
using namespace std;

#define MAX 20
#define INF 9999

int n, adj[MAX][MAX];

void bfs(int start) {
    int q[MAX], front=0, rear=0, visited[MAX]={0};
    q[rear++] = start; visited[start]=1;
    cout << "BFS: ";
    while(front < rear) {
        int u = q[front++];
        cout << u << " ";
        for(int v=0; v<n; v++)
            if(adj[u][v] && !visited[v]) { q[rear++] = v; visited[v]=1; }
    }
    cout << "\n";
}

void dfsUtil(int u, int visited[]) {
    visited[u] = 1;
    cout << u << " ";
    for(int v=0; v<n; v++)
        if(adj[u][v] && !visited[v]) dfsUtil(v, visited);
}

void dfs(int start) {
    int visited[MAX]={0};
    cout << "DFS: ";
    dfsUtil(start, visited);
    cout << "\n";
}

int findMinKey(int key[], int mstSet[]) {
    int min=INF, min_index=-1;
```

Name: Dishita Bansal Roll No.1024030365
Batch:2C24

```
    for(int i=0;i<n;i++)
        if(!mstSet[i] && key[i]<min) { min=key[i]; min_index=i; }
    return min_index;
}

void primMST() {
    int key[MAX], parent[MAX], mstSet[MAX]={0};
    for(int i=0;i<n;i++) key[i]=INF;
    key[0]=0; parent[0]=-1;
    for(int count=0; count<n-1; count++) {
        int u = findMinKey(key, mstSet);
        mstSet[u]=1;
        for(int v=0;v<n;v++)
            if(adj[u][v] && !mstSet[v] && adj[u][v]<key[v]) { parent[v]=u;
key[v]=adj[u][v]; }
    }
    cout << "Prim MST:\n";
    for(int i=1;i<n;i++) cout << parent[i] << " - " << i << " = " <<
adj[i][parent[i]] << "\n";
}

int findParent(int parent[], int i) { return parent[i]==i ? i :
parent[i]=findParent(parent, parent[i]); }

void kruskalMST() {
    struct Edge{int u,v,w;} e[MAX*MAX];
    int eCount=0;
    for(int i=0;i<n;i++)
        for(int j=i;j<n;j++)
            if(adj[i][j]) { e[eCount].u=i; e[eCount].v=j;
e[eCount].w=adj[i][j]; eCount++; }
    for(int i=0;i<eCount-1;i++)
        for(int j=i+1;j<eCount;j++)
            if(e[i].w>e[j].w){ Edge t=e[i]; e[i]=e[j]; e[j]=t; }
    int parent[MAX]; for(int i=0;i<n;i++) parent[i]=i;
    cout << "Kruskal MST:\n";
    for(int i=0;i<eCount;i++){
        int u=findParent(parent,e[i].u),v=findParent(parent,e[i].v);
        if(u!=v){ cout << e[i].u << " - " << e[i].v << " = " << e[i].w <<
"\n"; parent[u]=v; }
    }
}

void dijkstra(int src) {
    int dist[MAX], visited[MAX]={0};
    for(int i=0;i<n;i++) dist[i]=INF;
    dist[src]=0;
    for(int count=0;count<n-1;count++){
```

Name: Dishita Bansal Roll No.1024030365
Batch:2C24

```
        int u=-1,minD=INF;
        for(int i=0;i<n;i++) if(!visited[i] && dist[i]<minD) { minD=dist[i];
u=i; }
        visited[u]=1;
        for(int v=0;v<n;v++)
            if(adj[u][v] && !visited[v] && dist[u]+adj[u][v]<dist[v])
                dist[v]=dist[u]+adj[u][v];
    }
    cout << "Dijkstra distances from " << src << ":\n";
    for(int i=0;i<n;i++) cout << i << ": " << dist[i] << "\n";
}

int main() {
    int ch, start;
    cout << "Enter number of vertices: "; cin >> n;
    cout << "Enter adjacency matrix:\n";
    for(int i=0;i<n;i++) for(int j=0;j<n;j++) cin >> adj[i][j];
    do {
        cout << "\n1.BFS 2.DFS 3.Prim 4.Kruskal 5.Dijkstra 6.Exit\nChoice: ";
        cin >> ch;
        switch(ch){
            case 1: cout << "Enter start vertex: "; cin >> start; bfs(start);
break;
            case 2: cout << "Enter start vertex: "; cin >> start; dfs(start);
break;
            case 3: primMST(); break;
            case 4: kruskalMST(); break;
            case 5: cout << "Enter source vertex: "; cin >> start;
dijkstra(start); break;
        }
    }while(ch!=6);
    return 0;
}
```

OUTPUT:

Name: Dishita Bansal Roll No.1024030365
Batch:2C24

```
Enter number of vertices: 4
Enter adjacency matrix:
0 1 3 0
1 0 1 4
3 1 0 2
0 4 2 0

1.BFS 2.DFS 3.Prim 4.Kruskal 5.Dijkstra 6.Exit
Choice: 1
Enter start vertex: 0
BFS: 0 1 2 3

1.BFS 2.DFS 3.Prim 4.Kruskal 5.Dijkstra 6.Exit
Choice: 2
Enter start vertex: 0
DFS: 0 1 2 3

1.BFS 2.DFS 3.Prim 4.Kruskal 5.Dijkstra 6.Exit
Choice: 3
Prim MST:
0 - 1 = 1
1 - 2 = 1
2 - 3 = 2

1.BFS 2.DFS 3.Prim 4.Kruskal 5.Dijkstra 6.Exit
Choice: 4
```

```
1.BFS 2.DFS 3.Prim 4.Kruskal 5.Dijkstra 6.Exit
Choice: 4
Kruskal MST:
0 - 1 = 1
1 - 2 = 1
2 - 3 = 2

1.BFS 2.DFS 3.Prim 4.Kruskal 5.Dijkstra 6.Exit
Choice: 5
Enter source vertex: 0
Dijkstra distances from 0:
0: 0
Dijkstra distances from 0:
0: 0
1: 1
2: 2
3: 4

1.BFS 2.DFS 3.Prim 4.Kruskal 5.Dijkstra 6.Exit
Choice:
```

THANK YOU